Godot 大体量程序化城市调研

Executive Summary

如果目标是“几十平方公里级”的 3D 城市，真正的核心问题通常不是“能不能把地图做大”，而是“能不能只让玩家附近的一小部分世界处于高成本状态”。结合 Godot 官方文档和经典程序化城市论文，比较稳妥的方案是：用分层数据先定义城市骨架（道路网、分区、街区、地块），再用分块流式加载、HLOD、MultiMesh、遮挡剔除和分块导航去承载运行时表现 [1][2][3][4][5][6][7]。

对 `godot_citys` 这种第三人称城市游戏来说，几十平方公里并不一定马上需要 Godot 双精度编译版。按常见约定如果 `1 单位 ~= 1 米`，例如 49 平方公里约等于 `7km x 7km`，把世界中心放在原点后，边界距离原点约 `3.5km`，仍明显落在 Godot 文档给出的第三人称单精度建议范围之内；这意味着你当前优先级应放在流式架构和内容分层，而不是先折腾全引擎双精度 [1]。这里的尺度判断是我基于官方精度范围做的工程推断。

Key Findings

- **大城市要先做“数据城市”，再做“可见城市”**：经典流程普遍先生成道路网络，再切街区/地块，最后生成建筑与细节，而不是一上来堆满几何体 [8][9][10]。
- **Godot 可以承载大场景，但不能把整座城当一个场景节点树长期常驻**：官方文档建议用后台资源加载、线程安全的 server API、MultiMesh、HLOD 和遮挡剔除来控制活跃成本 [2][3][4][5][11]。
- **真正的性能瓶颈通常在渲染、导航和节点规模，不是地图名义面积**：Godot 官方明确指出，大量实例、导航网格烘焙、SceneTree 解析、复杂网格来源都会直接拖垮帧率 [2][6][7][11]。
- **导航必须分块**：Godot 官方专门给了“大世界导航网格分块烘焙”的方案，包括 `baking bound` 和 `border_size` 来消除块边缝隙 [6]。
- **最推荐的城市生成方法是混合式**：主干路和分区用规则/语法生成，街道风格和局部纹理用 tensor field 或 example-based 方法做“风格控制”，这样既可控又不像纯随机拼积木 [8][9][10]。

Detailed Analysis

1. 大城市生成，先决定“骨架”再决定“细节”

Parish 与 Müller 的经典工作把流程拆成了非常清晰的三层：先根据土地、水域、人口等输入生成高速路和街道系统，再把土地切成 lots，最后才生成建筑几何 [8]。这套分层今天依然非常实用，因为它天然

适合“高层数据常驻，低层表现按需生成”。

Chen 等人在 2008 年把“街道生成”进一步做成了可编辑的 tensor field 驱动系统。它的价值不在于数学本身，而在于你可以非常明确地控制一片区域更像曼哈顿网格、欧洲斜街、河岸弯曲道路还是放射状主干道 [9]。这很适合“城市分区”的概念设计。

Aliaga 等人的 example-based urban layout synthesis 则提供了另一条路：从真实或手工设计的城市样本中提取结构与视觉特征，再合成新布局 [10]。如果你想让不同城区拥有显著风格差异，例如“老城区更密、科技园更规整、城郊更松散”，这种思路比纯 Perlin 噪声更接近真实城市。

工程上我更建议你采用混合架构：

- 主干路网和行政分区：规则化、可重现的种子生成。
- 街区内部道路和街角形态：tensor field 风格引导。
- 建筑退线、街景节奏、局部地块排布：example-based 或模板库加权选择。

这不是论文原文的逐字要求，而是我基于三类方法特点做的综合推荐。

2. Godot 里“几十平方公里”首先不是精度问题，而是流式问题

Godot 官方给出的单精度建议范围里，第三人称 3D 游戏在距离原点 `8192` 到 `16384` 单位前通常都还能避免明显渲染/物理异常 [1]。同时官方也说明，超过更大范围后，双精度 large world coordinates 或 origin shifting 才通常更有必要；而 large world coordinates 需要自行编译 `precision=double` 的编辑器和导出模板，不是你当前这套 Godot 4.6 默认二进制开箱即用的开关 [1]。

这带来一个关键结论：如果你的目标只是 `30-80 km²`，例如 `6km x 6km`、`7km x 7km` 或 `8km x 8km`，只要世界中心布局合理，你很可能还在单精度的舒适区。此时更重要的是把城市拆成 chunk，而不是急着重编引擎。这一判断是我结合 Godot 文档数值做的推断，不是文档原话。

对 `godot_citys`，我建议先按如下层级建模：

- `District`：`1km x 1km` 或 `512m x 512m` 级别，只存道路图、分区标签、种子。
- `BlockChunk`：推荐先试 `256m x 256m`。这是运行时加载/卸载的主单位。
- `Parcel`：街区内部地块，不必做成节点常驻，可按需数据化。
- `FacadeSet / PropSet`：只在近距离解包为高细节内容。

例如 `7km x 7km` 的城市，如果按 `256m` chunk 切分，大约是 `28 x 28 = 784` 个 chunk；但运行时你完全不需要常驻 784 个，只需围绕玩家保留 `3x3`、`5x5` 或“近中远三圈”窗口。这部分数字是工程估算。

3. 渲染规模要靠 MultiMesh、HLOD、遮挡剔除一起扛

Godot 官方对 MultiMesh 的表述非常直接：它可以在一次 draw primitive 里画出成千上万甚至百万级实例，但单个实例没有独立屏幕/视锥剔除，整个 MultiMesh 只能“整片可见或整片不可见”，所以官方建议把它们拆分成不同世界区域 [2]。这意味着树木、路灯、护栏、停车位、窗格、重复性广告牌这类资产，非常适合“按 chunk 拆分的 MultiMesh”。

与此同时，Godot 的 Visibility Ranges / HLOD 明确支持 `MeshInstance3D` 和 `MultiMeshInstance3D`，并提供 `Begin/End`、fade mode 和 `Visibility Parent` 来切换近景单体网格与远景合批网格 [4]。这很适合做：

- 近景：独立建筑、可交互门店、真实碰撞。
- 中景：整街面合批网格。
- 远景：整街区轮廓或 skyline proxy。

官方还建议远距离 LOD 使用更简单的材质，并指出 dithering 过渡往往比 alpha fade 更高效 [4]。

再往上叠一层，Godot 的 occlusion culling 支持 `ArrayOccluder3D`，官方明确说它对脚本程序化生成很有用 [3]。对大城市而言，这非常关键，因为街区、楼排、围墙天然就是遮挡体。换句话说，你完全可以在生成 block 时顺手生成一组粗略遮挡体，而不是等美术后期再手工布。

4. 流式加载和线程模型必须避开 SceneTree 瓶颈

Godot 官方建议用 `ResourceLoader.load_threaded_request`、`load_threaded_get_status` 和 `load_threaded_get` 去做后台资源加载 [5]。同时官方在 thread-safe APIs 里强调：活动中的 SceneTree 不是线程安全的；如果你要在线程里准备 chunk，应该在线程里生成数据或离树场景，再回主线程 `call_deferred` 挂载；如果真要大规模并行，更安全的是直接使用 server API [11]。

这条对程序化城市非常重要，因为“生成大城市”很多时候不是真的算不动，而是你把太多 Node3D、MeshInstance3D、CollisionShape3D 在主线程里连续创建，结果导致卡顿。官方也特别提到，Global Scope singletons 和一些 server API 适合在线程里管理成千上万实例 [11]。

因此比较合理的运行时分工是：

- 工作线程：决定哪些 chunk 进入/退出，计算道路图、地块、实例变换、导航源数据。
- 主线程：只负责挂载少量场景、交换可见状态、应用已经准备好的结果。
- RenderingServer / MultiMesh / NavigationServer：承担批量底层状态更新。

5. 导航和 AI 不能“整城一张 navmesh”

Godot 官方不仅有导航优化页，还专门有“大世界导航网格分块烘焙”章节 [6][7]。官方指出，导航性能与 polygon / edge 数量直接相关，而不是简单与地图面积相关；如果世界被切成过多微小多边形，路径搜索性能会显著下降 [7]。同时官方明确建议：

- runtime bake 尽量走后台线程 [7]
- source geometry 尽量用简化过的碰撞体，而不是复杂视觉网格 [6][7]
- 如果必须动态烘焙大世界，优先用 procedural arrays 作为 source geometry，这样整个流程更容易放到后台线程 [6]
- chunk 边界用 `baking bound` + `border_size` 去对齐 [6]

这意味着城市 NPC 的导航最好分三层：

- 远距离宏观层：道路图或 lane graph，只做 district-to-district / block-to-block 规划。
- 近距离中观层：chunk navmesh，用于街区内部移动。
- 极近距离微观层：局部避障与人群扰动。

这部分分层是我基于 Godot 官方导航建议做的架构推断。

6. 对 `godot_citys` 的具体推荐

如果目标是把当前原型扩到“几十平方公里级城市”，我建议按下面顺序推进：

1. **先做城市数据模型，不先做美术规模**
   - 世界种子
   - district graph
   - arterial / secondary road graph
   - block / parcel 数据
2. **再做 chunk streamer**
   - `256m` chunk 起步
   - 玩家周围 `5x5` 活跃
   - 近/中/远三层可见策略
3. **再做实例化资产体系**
   - 路灯、树、护栏、停车位、窗户、路牌全部 chunk-local MultiMesh
   - 建筑远景合批 mesh
   - block 自动生成 `ArrayOccluder3D`
4. **最后再引入动态系统**
   - 车辆 lane graph
   - pedestrian spawn zones

- 商店/任务/警戒区等交互系统

推荐原因很简单：只有先把“世界如何被切块、调度、降级”这件事做对，后面不管加车流、NPC、任务还是 AI agent，都不会把底盘打爆。

Areas of Consensus

- 大城市生成需要分层表示：道路网、地块、建筑不要一次性做成最终几何 [8][9][10]。
- 大世界运行时必须 chunk 化，不能整城常驻高成本节点树 [2][5][6][11]。
- 远景一定要降级：HLOD、远景合批、简化材质、遮挡剔除是标配 [2][3][4]。
- 导航必须按 chunk 或区域控制复杂度，不能无限细碎化 [6][7]。

Areas of Debate

- **道路生成范式**：规则语法、tensor field、example-based 都能成立；取舍主要看你是更重“艺术可控”还是更重“真实感迁移” [8][9][10]。
- **双精度还是 origin shifting**：Godot 官方同时承认两条路线都可行，但双精度需要自编译且有兼容/性能代价，origin shifting 则增加逻辑复杂度 [1]。
- **节点树还是 server API 为主**：Godot 官方没有要求你全走底层 API，但在超大规模实例下，server API 明显更适合高吞吐 [2][11]。

Sources

[1] Godot Engine Documentation. “Large world coordinates.” 官方文档，适合判断单精度范围、双精度编译要求与 origin shifting 取舍。
https://docs.godotengine.org/en/stable/tutorials/physics/large_world_coordinates.html

[2] Godot Engine Documentation. “Optimization using MultiMeshes.” 官方文档，说明 MultiMesh 的能力、限制与按区域拆分的必要性。
https://docs.godotengine.org/en/stable/tutorials/performance/using_multimesh.html

[3] Godot Engine Documentation. “Occlusion culling.” 官方文档，指出 `ArrayOccluder3D` 适合脚本化程序生成。
https://docs.godotengine.org/en/stable/tutorials/3d/occlusion_culling.html

[4] Godot Engine Documentation. “Visibility ranges (HLOD).” 官方文档，说明 Godot 的 HLOD、fade mode 和 visibility parent 工作方式。
https://docs.godotengine.org/en/stable/tutorials/3d/visibility_ranges.html

[5] Godot Engine Documentation. “Background loading.” 官方文档，给出后台线程资源加载接口。
https://docs.godotengine.org/en/stable/tutorials/io/background_loading.html

[6] Godot Engine Documentation. “Using navigation meshes.” 官方文档，包含 large worlds 的导航 chunk 烘焙方案。
https://docs.godotengine.org/en/stable/tutorials/navigation/navigation_using_navigationmeshes.html

[7] Godot Engine Documentation. “Optimizing Navigation Performance.” 官方文档，说明导航性能与 source geometry、polygon/edge 数量、同步成本的关系。
https://docs.godotengine.org/en/stable/tutorials/navigation/navigation_optimizing_performance.html

[8] Parish, Yoav I. H., and Pascal Müller. “Procedural Modeling of Cities.” SIGGRAPH 2001. 经典奠基论文，提出道路网 -> 地块 -> 建筑的城市程序化管线。
https://www.researchgate.net/publication/220720591_Procedural_Modeling_of_Cities

[9] Chen, Guoning, Gregory Esch, Peter Wonka, Pascal Mueller, and Eugene Zhang. “Interactive Procedural Street Modeling.” ACM Transactions on Graphics / SIGGRAPH 2008. 经典街道生成论文，强调 tensor field 驱动的可控道路网络。
https://www.sci.utah.edu/~chengu/street_sig08/street_project.htm

[10] Aliaga, Daniel G., Carlos A. Vanegas, and Bedrich Benes. “Interactive Example-Based Urban Layout Synthesis.” SIGGRAPH Asia 2008. 示例驱动的城市布局合成方案，适合不同城区风格迁移。
https://www.cs.purdue.edu/cgvlab/www/publications/Aliaga08ToG/

[11] Godot Engine Documentation. “Thread-safe APIs.” 官方文档，说明 SceneTree、resource loading、多线程和 server API 的边界。
https://docs.godotengine.org/en/stable/tutorials/performance/thread_safe_apis.html

Gaps and Further Research

- 这次调研主要覆盖“城市生成管线”和“Godot 运行时承载方式”，没有深入到车辆交通仿真、行人社会模拟、地铁/高架/立交层级交通。
- 还没有对 Godot 4.6 在你这台 Windows 11 机器上的真实 draw call、导航烘焙耗时、MultiMesh 更新吞吐做基准测试。
- 如果后续你要做“真正能开车高速穿城”的体验，还需要专门调研车辆物理、streaming 预取半径和道路 lane graph 数据结构。